# Ejercicios Resueltos

Conectando la aplicación al mundo: JDBC y Base de Datos
Desarrollo Orientado a Objetos II — Experiencia 3, Semana 7

---

10 ejercicios resueltos, cada uno con una temática distinta relacionada con JDBC, PreparedStatement, ResultSet y persistencia de datos en MySQL. Uno de los ejercicios implementa un CRUD completo e incluye una vista previa de cómo debería verse la pantalla del formulario.

# Ejercicio 1 — Clase de conexión reutilizable (ConexionBD)

**Temática:** Configuración básica de una conexión JDBC hacia MySQL.

**Enunciado:** Crea una clase `ConexionBD` que centralice los datos de acceso a la base de datos `verduleria` y exponga un método estático que retorne un objeto `Connection` listo para usar.

## Solución

```
package util;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexionBD {

    private static final String URL = "jdbc:mysql://localhost:3306/verduleria";
    private static final String USUARIO = "root";
    private static final String CONTRASENA = ""; // ajustar según entorno local

    public static Connection obtenerConexion() throws SQLException {
        return DriverManager.getConnection(URL, USUARIO, CONTRASENA);
    }
}
```

**Explicación:** El método `obtenerConexion()` es estático porque no depende del estado de una instancia; cualquier clase del proyecto puede invocarlo sin crear un objeto `ConexionBD`. Si el servidor MySQL no está activo o las credenciales son incorrectas, se lanza una `SQLException` que debe capturarse en la clase que consuma este método.

# Ejercicio 2 — Insertar un registro con PreparedStatement

**Temática:** Sentencia INSERT parametrizada.

**Enunciado:** Implementa un método que reciba un objeto `Producto` y lo guarde en la tabla `productos`, evitando la inyección SQL.

## Solución

```
public void agregarProducto(Producto producto) {
    String sql = "INSERT INTO productos(nombre, categoria, stock, valor) " +
                 "VALUES (?, ?, ?, ?)";

    try (Connection conn = ConexionBD.obtenerConexion();
         PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setString(1, producto.getNombre());
        stmt.setString(2, producto.getCategoria());
        stmt.setInt(3, producto.getStock());
        stmt.setInt(4, producto.getValor());

        stmt.executeUpdate();

    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

**Explicación:** Cada símbolo ? es reemplazado mediante los métodos `setString` / `setInt`, lo que impide que un valor ingresado por el usuario altere la estructura de la consulta SQL (inyección SQL). El bloque `try-with-resources` asegura que la conexión y el statement se cierren automáticamente.

# Ejercicio 3 — Consultar registros con ResultSet

**Temática:** Lectura de datos (SELECT) y recorrido de un ResultSet.

**Enunciado:** Implementa un método que consulte todos los productos y los cargue en un `DefaultTableModel` para mostrarlos en una JTable.

## Solución

```
public void cargarProductosDesdeBD(DefaultTableModel model) {
    String sql = "SELECT * FROM productos";

    try (Connection conn = ConexionBD.obtenerConexion();
         PreparedStatement stmt = conn.prepareStatement(sql);
         ResultSet rs = stmt.executeQuery()) {

        model.setRowCount(0); // limpiar tabla antes de recargar

        while (rs.next()) {
            model.addRow(new Object[]{
                rs.getInt("id"),
                rs.getString("nombre"),
                rs.getString("categoria"),
                rs.getInt("stock"),
                rs.getInt("valor")
            });
        }

    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

**Explicación:** `rs.next()` avanza fila por fila del resultado; mientras retorne `true` hay datos disponibles. Los métodos `getInt` y `getString` deben coincidir con el tipo de dato real de la columna en la base de datos.

# Ejercicio 4 — Actualizar un registro (UPDATE)

**Temática:** Modificación de datos existentes.

**Enunciado:** Implementa un método que actualice nombre, categoría, stock y valor de un producto identificado por su `id`.

## Solución

```
public void editarProducto(int id, Producto producto) {
    String sql = "UPDATE productos SET nombre=?, categoria=?, stock=?, valor=? " +
                 "WHERE id=?";

    try (Connection conn = ConexionBD.obtenerConexion();
         PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setString(1, producto.getNombre());
        stmt.setString(2, producto.getCategoria());
        stmt.setInt(3, producto.getStock());
        stmt.setInt(4, producto.getValor());
        stmt.setInt(5, id);

        int filasAfectadas = stmt.executeUpdate();
        System.out.println("Filas actualizadas: " + filasAfectadas);

    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

**Explicación:** `executeUpdate()` retorna un entero con la cantidad de filas afectadas; es útil para validar que la actualización realmente encontró y modificó un registro (si retorna 0, el `id` no existía).

# Ejercicio 5 — Eliminar un registro (DELETE)

**Temática:** Eliminación de datos con confirmación del usuario.

**Enunciado:** Implementa la eliminación de un producto por `id`, solicitando confirmación antes de ejecutar la sentencia DELETE.

## Solución

```
private void eliminarProducto(int id, DefaultTableModel model, int filaTabla) {
    int confirmacion = JOptionPane.showConfirmDialog(
        this,
        "¿Estás seguro de que deseas eliminar este producto?",
        "Confirmar eliminación",
        JOptionPane.YES_NO_OPTION
    );

    if (confirmacion == JOptionPane.YES_OPTION) {
        String sql = "DELETE FROM productos WHERE id=?";

        try (Connection conn = ConexionBD.obtenerConexion();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setInt(1, id);
            stmt.executeUpdate();
            model.removeRow(filaTabla);

        } catch (SQLException e) {
            e.printStackTrace();
            JOptionPane.showMessageDialog(this, "Error al eliminar el producto.");
        }
    }
}
```

**Explicación:** Solicitar confirmación con `JOptionPane.showConfirmDialog` evita eliminaciones accidentales. La fila se remueve de la tabla visual (`model.removeRow`) solo después de que la sentencia SQL se ejecuta correctamente, manteniendo la vista sincronizada con la base de datos.

# Ejercicio 6 — Manejo de excepciones al fallar la conexión

**Temática:** Manejo de errores JDBC (SQLException).

**Enunciado:** Implementa un método `main` que pruebe la conexión a la base de datos y muestre un mensaje claro si el servidor MySQL está apagado o las credenciales son incorrectas.

## Solución

```
public class Main {
    public static void main(String[] args) {
        try (Connection conn = ConexionBD.obtenerConexion()) {
            System.out.println("Conexión exitosa a la base de datos.");
        } catch (SQLException e) {
            System.err.println("Error al conectar con la base de datos:");
            System.err.println("Código: " + e.getErrorCode());
            System.err.println("Mensaje: " + e.getMessage());
        }
    }
}
```

**Explicación:** El bloque `try-with-resources` garantiza el cierre de la conexión aunque ocurra un error. Los campos `getErrorCode()` y `getMessage()` de `SQLException` permiten diagnosticar si el problema es de credenciales, de red o de configuración del driver.

# Ejercicio 7 — CRUD completo de la entidad Cliente (con vista de formulario)

**Temática:** Implementación de un CRUD completo con interfaz Swing.

**Enunciado:** La tienda necesita un módulo para gestionar clientes (nombre, correo, teléfono y categoría). Implementa las operaciones agregar, editar, eliminar y listar, conectadas a una interfaz gráfica con JTextField, JComboBox, JTable y JButton.

## 1. Script de la tabla en MySQL

```
CREATE TABLE clientes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    correo VARCHAR(100) NOT NULL UNIQUE,
    telefono VARCHAR(20),
    categoria VARCHAR(20) NOT NULL DEFAULT 'Regular'
);
```

## 2. Controlador con las 4 operaciones CRUD

```
public class ControladorClientes {

    public void agregarCliente(Cliente c) {
        String sql = "INSERT INTO clientes(nombre, correo, telefono, categoria) " +
                     "VALUES (?, ?, ?, ?)";
        try (Connection conn = ConexionBD.obtenerConexion();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, c.getNombre());
            stmt.setString(2, c.getCorreo());
            stmt.setString(3, c.getTelefono());
            stmt.setString(4, c.getCategoria());
            stmt.executeUpdate();
        } catch (SQLException e) { e.printStackTrace(); }
    }

    public void editarCliente(int id, Cliente c) {
        String sql = "UPDATE clientes SET nombre=?, correo=?, telefono=?, categoria=? " +
                     "WHERE id=?";
        try (Connection conn = ConexionBD.obtenerConexion();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, c.getNombre());
            stmt.setString(2, c.getCorreo());
            stmt.setString(3, c.getTelefono());
            stmt.setString(4, c.getCategoria());
            stmt.setInt(5, id);
            stmt.executeUpdate();
        } catch (SQLException e) { e.printStackTrace(); }
    }

    public void eliminarCliente(int id) {
        String sql = "DELETE FROM clientes WHERE id=?";
        try (Connection conn = ConexionBD.obtenerConexion();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setInt(1, id);
            stmt.executeUpdate();
        } catch (SQLException e) { e.printStackTrace(); }
    }

    public void cargarTabla(DefaultTableModel model) {
```

```
            model.setRowCount(0);
            String sql = "SELECT * FROM clientes";
            try (Connection conn = ConexionBD.obtenerConexion();
                 PreparedStatement stmt = conn.prepareStatement(sql);
                 ResultSet rs = stmt.executeQuery()) {
                while (rs.next()) {
                    model.addRow(new Object[]{
                        rs.getInt("id"), rs.getString("nombre"),
                        rs.getString("correo"), rs.getString("telefono"),
                        rs.getString("categoria")
                    });
                }
            } catch (SQLException e) { e.printStackTrace(); }
        }
    }
```

## 3. Vista: cómo se debería ver la pantalla del formulario

La ventana principal `GestionClientes.java` se organiza en dos paneles: a la izquierda el formulario con los campos de texto, el combo de categoría y los botones CRUD; a la derecha la JTable con el listado de clientes (la columna ID permanece oculta, igual que en `TiendaVerduras.java`).

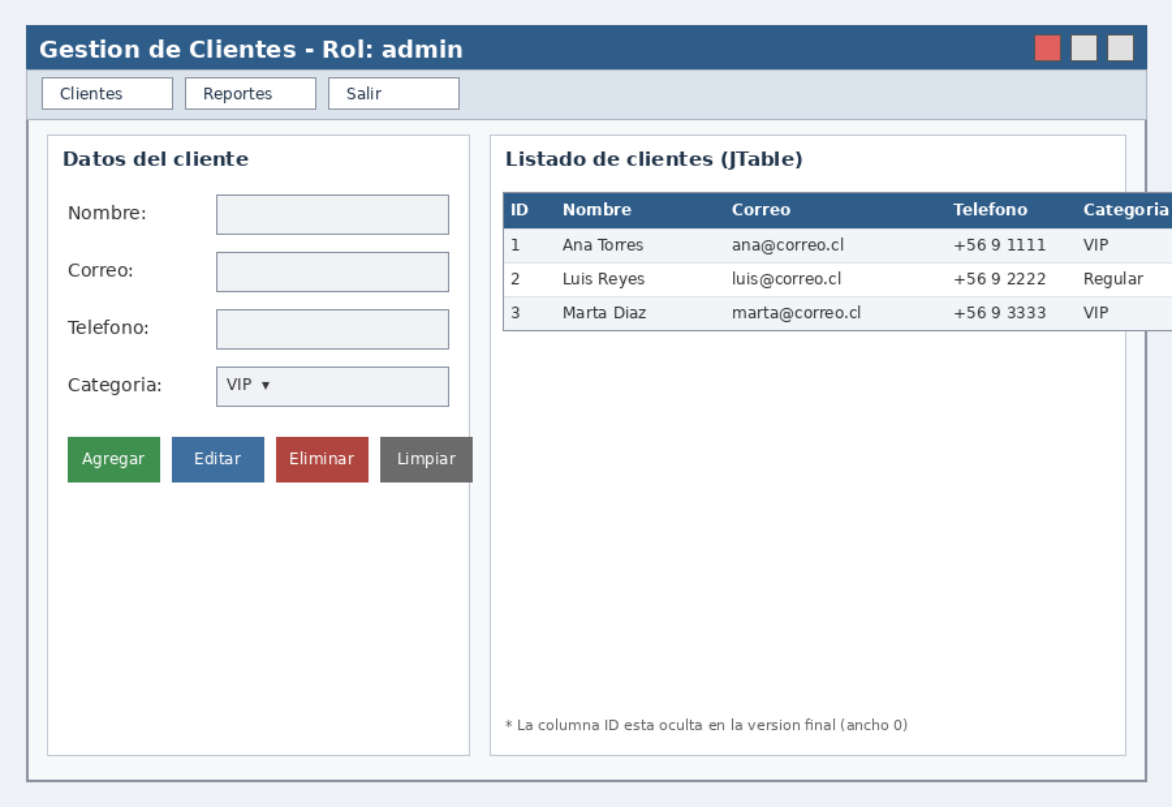


*Figura 1. Vista previa de la pantalla del formulario CRUD de clientes (mockup de referencia, no captura real de IntelliJ).*

## 4. Fragmento de la vista (eventos de los botones)

```
btnAgregar.addActionListener(e -> {
    Cliente nuevo = new Cliente(
        txtNombre.getText().trim(),
        txtCorreo.getText().trim(),
        txtTelefono.getText().trim(),
        (String) cmbCategoria.getSelectedItem()
    );
    controlador.agregarCliente(nuevo);
```

```
            controlador.cargarTabla(model);
            limpiarFormulario();
        });

        btnEliminar.addActionListener(e -> {
            int fila = tblClientes.getSelectedRow();
            if (fila >= 0) {
                int id = (int) model.getValueAt(fila, 0);
                controlador.eliminarCliente(id);
                controlador.cargarTabla(model);
            }
        });
```

**Explicación general:** El flujo sigue el mismo patrón visto durante la semana: la vista captura la acción del usuario, delega la operación al controlador (que usa PreparedStatement) y, al finalizar, recarga la tabla desde la base de datos para reflejar siempre el estado real persistido en MySQL.

# Ejercicio 8 — Cierre seguro de recursos con try-with-resources

**Temática:** Buenas prácticas en el manejo de conexiones.

**Enunciado:** Explica y corrige el siguiente código, que abre una conexión pero nunca la cierra correctamente si ocurre un error.

## Código con problema

```
// Versión incorrecta: si stmt.executeUpdate() lanza una excepción,
// conn.close() nunca se ejecuta y la conexión queda abierta.
Connection conn = ConexionBD.obtenerConexion();
PreparedStatement stmt = conn.prepareStatement(sql);
stmt.executeUpdate();
conn.close();
```

## Solución corregida

```
try (Connection conn = ConexionBD.obtenerConexion();
     PreparedStatement stmt = conn.prepareStatement(sql)) {

    stmt.executeUpdate();

} catch (SQLException e) {
    e.printStackTrace();
}
// conn y stmt se cierran automáticamente, incluso si hay una excepción.
```

**Explicación:** `try-with-resources` funciona con cualquier clase que implemente `AutoCloseable`, como `Connection`, `PreparedStatement` y `ResultSet`. Java cierra automáticamente los recursos declarados dentro del paréntesis, en orden inverso a como fueron abiertos, evitando fugas de conexiones.

# Ejercicio 9 — Mapeo de tipos de datos entre Java y SQL

**Temática:** Correspondencia de tipos al leer/escribir con JDBC.

**Enunciado:** Dada la siguiente tabla, indica y aplica el método correcto de `PreparedStatement` / `ResultSet` para cada columna.

| Columna SQL | Tipo SQL | Método PreparedStatement | Método ResultSet |
|---|---|---|---|
| `id` | `INT` | `setInt(pos, valor)` | `getInt("id")` |
| `nombre` | `VARCHAR(100)` | `setString(pos, valor)` | `getString("nombre")` |
| `activo` | `BOOLEAN` | `setBoolean(pos, valor)` | `getBoolean("activo")` |
| `valor` | `DECIMAL(10,2)` | `setDouble(pos, valor)` | `getDouble("valor")` |
| `fecha_registro` | `TIMESTAMP` | `setTimestamp(pos, valor)` | `getTimestamp("fecha_registro")` |

## Aplicación en código

```
String sql = "INSERT INTO ventas(id_cliente, valor, activo, fecha_registro) " +
             "VALUES (?, ?, ?, ?)";

try (Connection conn = ConexionBD.obtenerConexion();
     PreparedStatement stmt = conn.prepareStatement(sql)) {

    stmt.setInt(1, idCliente);
    stmt.setDouble(2, valorVenta);
    stmt.setBoolean(3, true);
    stmt.setTimestamp(4, new Timestamp(System.currentTimeMillis()));

    stmt.executeUpdate();

} catch (SQLException e) {
    e.printStackTrace();
}
```

**Explicación:** Usar el método equivocado (por ejemplo, `setString` para una columna `DECIMAL`) puede generar errores de conversión o resultados inesperados. Siempre se debe hacer coincidir el tipo de dato Java con el tipo de columna definido en la base de datos.

# Ejercicio 10 — Separación por capas: controlador delgado vs. lógica SQL directa

**Temática:** Buenas prácticas de arquitectura (preparación al patrón DAO).

**Enunciado:** Refactoriza un controlador que mezcla validación, SQL y actualización de la interfaz en un único método, separando responsabilidades.

## Antes: todo mezclado en un solo método

```
public void agregarProducto(String nombre, int stock, int valor,
                            DefaultTableModel model) {
    if (nombre.isEmpty() || stock <= 0 || valor <= 0) {
        JOptionPane.showMessageDialog(null, "Datos inválidos");
        return;
    }
    String sql = "INSERT INTO productos(nombre, stock, valor) VALUES (?, ?, ?)";
    try (Connection conn = ConexionBD.obtenerConexion();
         PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, nombre);
        stmt.setInt(2, stock);
        stmt.setInt(3, valor);
        stmt.executeUpdate();
        model.addRow(new Object[]{nombre, stock, valor});
    } catch (SQLException e) { e.printStackTrace(); }
}
```

## Después: responsabilidades separadas

```
// Vista: solo valida y recolecta datos del formulario
private void agregarProducto() {
    if (!validarFormulario()) return;
    Producto p = new Producto(txtNombre.getText(), stock, valor);
    controlador.agregarProducto(p);
    controlador.cargarProductosDesdeBD(model);
    limpiarFormulario();
}

// Controlador: solo delega al acceso a datos
public void agregarProducto(Producto p) {
    productoDAO.insertar(p);
}

// DAO: única capa que conoce SQL
public class ProductoDAOImpl implements ProductoDAO {
    public void insertar(Producto p) {
        String sql = "INSERT INTO productos(nombre, stock, valor) VALUES (?, ?, ?)";
        try (Connection conn = ConexionBD.obtenerConexion();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, p.getNombre());
            stmt.setInt(2, p.getStock());
            stmt.setInt(3, p.getValor());
            stmt.executeUpdate();
        } catch (SQLException e) { e.printStackTrace(); }
    }
}
```

**Explicación:** Separar la validación (vista), la coordinación (controlador) y el acceso a datos (DAO) reduce el acoplamiento: si cambia el motor de base de datos o la consulta SQL, solo se modifica la capa DAO, sin tocar la interfaz gráfica ni la lógica de negocio. Esta separación es la base del patrón DAO que se

profundiza en la Semana 8.

---